abr. 13, 2026

Orientação a Objetos

convidado Bruno Joaquim Cassia Dobler Fabio Diniz Francisco Schneider Gabriel Severo Gabriel kohlrausch Ruan Rodrigues aline.bianchini@t-systems.com ana.santos@t-systems.com analista.kuehn@outlook.com anderson.hilario@t-systems.com anderson.perim@t-systems.com andersongabriel_maria@hotmail.com anselmogabriel421@gmail.com bmf.pedro@gmail.com bruna.nery@t-systems.com bruno.destro@t-systems.com caique.rodrigues@t-systems.com d.silva@t-systems.com danilosilva2017@hotmail.com desenvolvedor@tidsoftware.com.br douglas.nishiyama@t-systems.com dpsnbsb79@gmail.com eduardo.kubota@t-systems.com eudy.amparo@t-systems.com fabio.marinelli@t-systems.com farley.t.i@hotmail.com felipeneves089@gmail.com gustavo.ferrazfontes@gmail.com hewerton.souza@t-systems.com i.parmigiani@t-systems.com igor.romulo.santos@gmail.com jeanderson.silva@t-systems.com jl.oliveir@gmail.com jonathan.jimenez@iatec.com karinetrevisann@gmail.com l.regioli@outlook.com leandro.piqueira@t-systems.com lucas.p.oliveira@outlook.pt lxsilvareis@gmail.com marcelohcordeiro@gmail.com marcio@almob.com.br marcos.ammon@gmail.com moraesguga@gmail.com n.segeti@t-systems.com nadson.dr11@gmail.com Paulo Scatena rafael.dias@t-systems.com regis.maioli@t-systems.com reinaldo.rodrigues@t-systems.com renan-silva@t-systems.com renan.baptista.cabral@gmail.com rodrigo_7sch@hotmail.com tatidornel@gmail.com thiago.fernandes@t-systems.com victor.prospero@t-systems.com victor.santos@t-systems.com victor.souza@t-systems.com vinicius.bonicio@t-systems.com webdeveloperrm@gmail.com wesley.boccaletto@t-systems.com wsilva@t-systems.com fabiano.felgas@gmail.com

Anexos Orientação a Objetos

Registros da reunião Transcrição Gravação

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

Resumo

A aula sobre orientação a objetos abordou lacunas comuns, questionando a abertura de classes, discutindo Value Objects e focando em princípios SOLID e padrões de tratamento de falhas.

Classes Abertas e Encapsulamento
Foi discutida a insegurança de classes com sets abertos, destacando que o encapsulamento deve ser priorizado em métodos que representem operações de negócio. O encapsulamento ideal deve centralizar regras e evitar estados inválidos do objeto.

Herança, Composição e Value Objects
A herança foi contrastada com a composição, que é preferível para evitar problemas de runtime e risco do princípio de Liskov. Value Objects (usando struct e record) foram sugeridos para evitar a obsessão por tipos primitivos e agregar comportamento ao dado.

SOLID, Exceções e Result Pattern
Os 5 princípios SOLID foram revisados, enfatizando a Inversão de Dependência e a criação de interfaces segregadas. O Result Pattern foi apresentado como alternativa às Exceções para tornar métodos mais semânticos, facilitando o tratamento de falhas e a legibilidade do código.

Próximas etapas

- [ ] [O grupo] Publicar Informações DDD: Fornecer detalhes adicionais sobre o curso DDD no grupo. Compartilhar informações mais detalhadas sobre o curso.
- [ ] [O grupo] Revisar Exceções: Atualizar código que relança exceções; garantir que a cadeia completa da exceção seja preservada usando throw;.
- [ ] [O grupo] Consultar Diretiva: Pesquisar e seguir a diretiva Microsoft CA2201.

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

- [ ] [O grupo] Estender Discussão: Estender discussão sobre princípios SOLID. Continuar debate em grupo no WhatsApp.

Detalhes

- **Apresentação e Escopo da Aula**: Francisco Schneider iniciou a sessão dando as boas-vindas e confirmou que seria o instrutor responsável pela aula atual e por alguns encontros futuros, abordando o tema de orientação a objetos. A intenção não é começar do zero, mas sim discutir pontos de orientação a objetos que são percebidos como lacunas no dia a dia com clientes, focando em conceitos que tragam sentido prático (00:02:59). O módulo está na fase inicial de nivelamento, e a aula deve ser uma conversa interativa, incentivando os participantes a interromperem, levantarem a mão e trazerem discussões (00:04:46).
- **Dúvida sobre o Curso DDD do Jeito Certo**: Márcio das Chagas Ramos perguntou a Francisco Schneider sobre a previsão de novas turmas online ou a modalidade de curso para o "DDD do Jeito Certo". Francisco Schneider informou que existem aulas gravadas (com Elemar e Fábio) e que, sazonalmente, são abertas turmas ao vivo com Elemar, mas não pôde confirmar a situação atual ou o lançamento de novas turmas (00:00:00). Francisco Schneider se comprometeu a pedir que a equipe colocasse mais informações sobre o DDD no grupo (00:02:59).
- **Contexto e Estruturas de Dados Simples**: Para introduzir a discussão, Francisco Schneider propôs um cenário simples envolvendo um aluno, uma turma e a matrícula do aluno nessa turma. O ponto de partida é questionar a prática comum observada nos clientes, onde as classes são majoritariamente públicas, com quase tudo sendo classe e frequentemente contendo *sets* abertos. O objetivo é questionar se a classe e o acesso público são a estrutura de dados mais adequada e se *sets* abertos fazem sentido (00:06:12).
- **Problemas na Instanciação da Matrícula**: Ao analisar a instanciação da classe `Matrícula`, Francisco Schneider pediu que os participantes identificassem problemas no código. Gustavo Ferraz Fontes apontou que a classe está acoplada com o resto do sistema devido aos *gets* e *sets* públicos, o que gera acoplamento com qualquer consumidor (00:07:39). Além disso, a implementação atual não garante nenhuma validação na construção do objeto, comprometendo a consistência dos dados sem um construtor mínimo (00:09:01).

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

- **Objeto Aberto e Inseguro**: Francisco Schneider concordou com as observações de Gustavo Ferraz Fontes, resumindo que a classe é um objeto aberto e inseguro com baixa coesão. É inseguro porque permite qualquer alteração dos dados sem controle, e é aberto porque não há garantia de que todas as propriedades necessárias para a inicialização sejam informadas (00:09:01). É necessário utilizar artifícios da linguagem para evitar que a classe atinja um estado inválido (00:10:20).
- ****Encapsulamento e *Trade-offs*****: O encapsulamento foi apresentado como uma forma de atacar na raiz o problema de objetos em estado inválido (00:11:51). Embora estruturas anêmicas e classes simples existam e tenham aplicação, é crucial questionar se essa simplicidade não tem o potencial de tornar a classe inválida. Francisco Schneider também mencionou que a premissa deve ser desconfiar do potencial de brechas que podem levar a erros (00:10:20).
- **Discussão sobre Implementações de Getter e Setter**: Francisco Schneider confrontou duas implementações para a classe `Person`: uma com `get; set;` automático e outra com métodos explícitos `getName()` e `setName()`. Gustavo Ferraz Fontes defendeu a segunda opção, pois permite encapsular o *set* e garantir a validade do dado por meio de um ponto único de alteração (00:11:51). Natan Barros observou que, sem validação real, a segunda forma parece ser a mesma coisa que a primeira, apenas com métodos (00:13:11).
- **Açúcar Sintático do C\# e Benefícios do Encapsulamento**: Francisco Schneider explicou que, no C\#, a primeira implementação (*get; set;*) é, no fim, açúcar sintático, gerando código muito similar (um *backing field* e métodos *get/set*) à segunda. Argumentou que a segunda implementação não é essencialmente melhor do que a primeira, uma vez que o C\# já resolve o encapsulamento (00:14:25). O ponto principal é que o encapsulamento, quando feito apenas por regra (*by the book*), pode gerar complexidade ou código extra sem trazer todos os benefícios (00:15:52).
- **Métodos Representando Operações de Negócio**: O encapsulamento ideal deve estar contido em métodos que representem uma operação de negócio significativa. Ao evitar o empilhamento de *sets* e *gets* e, em vez disso, expor operações de negócio, as regras de negócio são puxadas para dentro da própria classe, onde devem estar centralizadas (00:15:52). Francisco Schneider usou o exemplo de um método `Casar()` para alterar propriedades de estado civil e nome, aplicando as validações pertinentes internamente, em vez de deixar a responsabilidade e as regras de negócio externas à classe (00:18:41).

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

- **Discussão sobre o Padrão Builder**: Farley Souza levantou uma abordagem que utilizava o padrão Builder para realizar validações de negócio, mantendo o modelo de domínio (a classe `Person` com *gets* e *sets* normais) para expressar o modelo (00:21:01). Francisco Schneider esclareceu que o Builder é um padrão criacional útil para delegar a criação de objetos, especialmente quando há várias maneiras distintas de fazê-lo, mas não é adequado para a alteração do estado de um objeto já construído. A alteração de estado deve ser feita preferencialmente por métodos expressos na classe de domínio (00:22:16).
- ****Melhorias na Classe `Matrícula` e Regra Geral para Sets****: Em uma versão melhorada da classe `Matrícula`, a operação de alteração de turma foi encapsulada em um método de negócio, e foi estabelecido um construtor para garantir a validação inicial do CPF (00:23:36). Francisco Schneider compartilhou a prática de criar estruturas sem *set* aberto (apenas *get*), pois é mais fácil abrir o acesso ao mundo (adicionando *private set* e, se necessário, *set* público) do que tentar remover *sets* que já possuem referências externas (00:25:06).
- **Modelagem e Interface Pública do Objeto**: O foco da modelagem deve estar na interface pública do objeto, ou seja, as partes que permitem contato com o mundo externo (00:25:06). O padrão observado é que as classes são anêmicas e totalmente abertas; no entanto, o mais adequado é expandir a interface pública de forma gradativa e apenas com o necessário, garantindo que o estado da estrutura seja alterado internamente de forma controlada (00:26:26). O benefício disso é a centralização das regras de negócio, maior estabilidade e testabilidade do código (00:27:50).
- **Herança e Polimorfismo**: Dando continuidade, Francisco Schneider introduziu a herança e o polimorfismo, utilizando como plano de fundo uma estrutura de `Turma` e suas especializações (`TurmaFutebol`, `TurmaXadrez`) (00:27:50). A regra simples para verificar se a herança se encaixa é o teste "é um" (`is-a`) entre a classe derivada e a classe base (00:29:15). Foi alertado que a herança pode tornar o código mais complexo do que a composição, especialmente com a possibilidade de cadeias de herança no C\# (00:30:40).
- **Preferência por Composição e Liskov**: Francisco Schneider expressou preferência por trabalhar com composição sempre que possível, embora a herança seja um recurso válido e um pilar de orientação a objetos (00:30:40). O polimorfismo garante que a instância concreta da classe (`TurmaFutebol`, `TurmaNatação`, etc.) terá seu método específico acionado em tempo de execução, mesmo quando referenciada pela classe abstrata (00:31:56). Gustavo Ferraz Fontes e sua equipe preferem a composição, pois ela tende a

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

gerar menos problemas de *runtime* do que a herança, citando o princípio de Liskov como um risco quando há extensões de herança (00:48:39).

- **Obsessão por Tipos Primitivos e Value Objects**: Foi discutida a "obsessão por tipos primitivos" (string, long, double, etc.), onde classes grandes e anêmicas utilizam muitos primitivos, muitas vezes sendo uma cópia das colunas do banco de dados. Essa prática empurra as regras de negócio para fora da classe (00:33:28). A composição, por sua vez, permite unir o comportamento ao dado, como exemplificado pela extração de propriedades de endereço em um Value Object (00:35:01).
- **Value Objects Modernos em C\#**: Para a criação de Value Objects, a prática moderna em C\# utiliza `struct` (que garante comparação propriedade a propriedade) e `record` (que oferece açúcar sintático) (00:35:01). Francisco Schneider sugeriu a fórmula `public readonly struct record` como uma boa maneira de trabalhar com Value Objects imutáveis (00:36:35). Gustavo Ferraz Fontes comentou que a equipe dele tem usado bastante `record` e `readonly` em novas soluções, mas que a classe ainda é a mentalidade predominante, sendo necessário um esforço cultural para nivelar o uso de `record` e `struct` (00:37:58).
- **Vantagens de Usar o Tipo Correto**: Francisco Schneider enfatizou que o uso correto de tipos (como `struct` ou `record` para Value Objects) é basilar e pode trazer ganhos importantes de performance e alocação de memória. Um exemplo de Value Object como `CPF` foi apresentado, utilizando encapsulamento (*field* privado), imutabilidade (`readonly`) e acesso ao valor por meio de `ToString()` (00:39:28) (00:41:33).
- **Operadores Implícitos e Padrão Idiomático**: Francisco Schneider destacou que operadores implícitos (como no Value Object `CPF`) podem ser uma ferramenta útil, especialmente para modernizar um domínio legado, permitindo a conversão automática de tipos (ex: `string` para `CPF`) sem quebrar diversas referências externas (00:42:53). Além disso, é importante se aproximar do padrão idiomático da linguagem (ex: usar `Parse` e `TryParse`) para facilitar a curva de aprendizado de novos desenvolvedores (00:44:27).
- **Genéricos e Reutilização de Código**: Genéricos foram introduzidos como um recurso antigo, mas pouco explorado, que permite a reutilização de código. Eles são utilizados em diversas estruturas do framework (Listas, Tuplas, Actions, Functions) (00:52:36). Algumas restrições úteis de genéricos foram apresentadas, como o uso de *where* para garantir que o tipo implemente uma interface (00:54:06).
- **Restrições Específicas em Genéricos**: Francisco Schneider detalhou restrições mais específicas em genéricos: a restrição `new()` obriga que o

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

tipo genérico possua um construtor vazio, o que é importante para cenários de ORM, por exemplo (00:54:06). A restrição `struct` (não nulo) garante que o tipo não pode ser anulável (00:55:29). O uso de *covariance* (com a palavra-chave `out` em interfaces ou com tipos como `IEnumerable`) garante que uma lista de um tipo derivado possa ser passada como uma lista do tipo base, desde que o método não altere a lista (00:57:06).

- **Conceitos de Contravariância e Covariância**: Francisco Schneider explicou que o conceito de contravariância é relevante ao trabalhar com genéricos, especialmente em cenários de herança e tipos derivados, apresentando um exemplo de método que lida com o tipo base `Employee` (01:00:04). A covariância auxilia em abstrações, permitindo o uso do compilador a favor do código, enquanto a contravariância permite o trabalho com herança e tipos mais ou menos derivados. O uso de genéricos não se limita ao nível de classe, sendo também possível em métodos dentro de classes simples (01:01:38).
- **Uso de Genéricos na Prática**: Gustavo Ferraz Fontes descreveu o uso de genéricos para criar *specifications* (especificações) na construção de *queries* para obtenção de registros no banco de dados. Isso permite a reutilização de código, a nomeação e o comportamento das *queries*, e facilita a testabilidade, como a criação de testes de unidade (01:03:10). Lucas Pereira mencionou o uso de genéricos para o reaproveitamento de código, como na construção de *consumers* e *frameworks* internos, onde apenas a classe ou objeto de resposta precisa ser alterado (01:04:26).
- **Genéricos e Otimização de Performance**: O uso de genéricos otimiza a performance por evitar *boxing* e *unboxing* desnecessários, pois o tipo é especificado em tempo de compilação (01:06:36). Essa otimização é crucial em escala, onde *boxing* e *unboxing* podem ser extremamente custosos. Além dos benefícios de reutilização de código, o uso de genéricos garante melhor performance (01:07:51).
- ****O Papel das Exceções (*Exceptions*)****: Houve um debate sobre se as exceções são inerentemente ruins, com a conclusão de que elas não são sempre ruins, mas dependem do contexto. Exceções podem sobrecarregar a memória e o processamento se usadas em excesso, mas são consideradas aceitáveis para lidar com violações de regras de domínio ou validações de negócio conhecidas (01:19:36). Jonathan Jimenez e Natan Barros preferem usar validações com *Result* ou *notification* para regras de negócio e reservar exceções apenas para casos verdadeiramente excepcionais, devido ao impacto na performance (01:20:53).

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

- **Tratamento Idiomático de Falhas em C#**: O C\# trata falhas nativamente por meio de exceções, sendo este o padrão idiomático da linguagem. Ao trabalhar com exceções, é recomendado contornar aquelas com alta probabilidade de ocorrência e buscar métodos alternativos, como o uso de `.Trys`, para evitar o lançamento de exceções desnecessárias (01:22:12). A premissa é projetar classes para que clientes possam evitar exceções, da mesma forma que as classes nativas do *framework* oferecem ferramentas para tal (01:23:56).
- **Formas de Relançar Exceções**: A maneira correta de relançar exceções para preservar a completude, incluindo todo o *stack trace* e *inner exceptions*, é usando apenas `throw`, sem especificar a exceção (ex: `throw ex`) (01:25:23). O uso de `throw ex` descarta toda a cadeia de exceções, preservando apenas a última, o que resulta em perda de contexto. Se for necessário adicionar uma nova exceção à cadeia, deve-se criar uma nova exceção e passar a cadeia existente como parâmetro (01:27:03).
- **Performance e Criação de Exceções Personalizadas**: A ideia de que exceções causam problemas de performance é considerada uma meia-verdade; embora sejam objetos pesados e ricos em informação, o *framework* otimizou o lançamento de exceções, e o impacto é geralmente imperceptível em versões modernas do C\# (01:31:42). Criar exceções personalizadas, como uma `Domain Exception`, é uma boa prática se houver a necessidade de um tratamento diferenciado, como mapear o tipo de erro para um código de status HTTP específico (ex: 422 para erros de negócio) (01:28:38) (01:34:31). O critério para criar um novo tipo de exceção é se o problema exigirá uma nova forma de tratamento (01:35:37).
- **Boas Práticas e Diretivas para Exceções**: Francisco Schneider enfatizou que não é uma boa prática lançar exceções com tipos genéricos, como `System.ApplicationException` (obsoleto) ou `System.Exception`, nem lançar exceções reservadas como `NullReferenceException` (01:36:41). Recomenda-se seguir a diretiva CA2201 da Microsoft, que oferece boas práticas sobre o que fazer e não fazer em relação a exceções (01:39:27). Métodos estáticos como `ThrowIfNullOrEmpty` devem ser priorizados em vez de usar construções `if...throw new` para checar argumentos (01:40:52).
- **Implementação de Exceções Personalizadas**: Ao criar exceções personalizadas, elas devem herdar de `Exception` e implementar os três construtores padrões para seguir o padrão idiomático da linguagem (01:40:52). Deve-se criar uma exceção personalizada somente se nenhuma existente no *framework* for adequada e se o novo tipo exigir um tratamento diferente. Exceções personalizadas podem ser enriquecidas com propriedades customizadas e ter construtores adicionais (01:42:15).

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

- **Validação de Domínio e Exceções**: A validação de dados deve ocorrer em várias camadas (front, controller/comando e domínio), sendo o domínio a parte mais interna e que não pode confiar nas camadas superiores para validação. Se a aplicação fosse ter apenas um mecanismo de proteção, ele deveria estar no domínio, garantindo que objetos não sejam criados em um estado inválido (01:45:15) (01:49:11). O domínio não deve ser "simpático", mas sim a última linha de defesa, e é o local onde o lançamento de exceções seria mais justificável (01:48:03).
- **Estratégias de Validação e *Tradeoffs***: A duplicação da regra de validação entre camadas (como comando e domínio) pode ser evitada se a regra for encapsulada em uma estrutura centralizada (como uma estrutura `CPF` ou `CNPJ`) e reutilizada, em vez de espalhar o código de validação (01:50:27). O consenso é que é preferível ter validação em múltiplas camadas do que ter um domínio exposto ou pouco protegido (01:53:02). Há um *tradeoff* entre a conveniência de não validar no domínio e o risco de exposição, versus a garantia de um domínio mais robusto, mas com código potencialmente mais verboso (01:58:15).
- **Discussão sobre Validações no Domínio e Camada de Borda**: Francisco Schneider e Gustavo Ferraz Fontes discutiram a importância de aplicar validações de negócios no domínio, que consideram o lugar mais severo para validações (02:00:58). Eles concordaram que uma boa rede de validação, incluindo validações na camada de borda, é essencial para garantir a integridade do objeto antes da criação. A validação na borda é importante para evitar problemas e uso desnecessário de recursos computacionais por causa de validações mais simples (01:59:33) (02:02:00).
- **Abordagem da API e Conceito de Domínio como API**: Gustavo Ferraz Fontes comparou o domínio com o conceito de API, no sentido da orientação a objetos, onde se deve expor apenas os dados necessários para o cliente e esconder os detalhes de implementação (01:59:33). Ele enfatizou que as validações mais rigorosas devem estar no domínio, que é o local único para essas verificações, como validar a duplicidade de um CPF na base de dados. Francisco Schneider concordou que o conceito de "Fest Validation" (validação na borda) é crucial para garantir a série de benefícios e evitar problemas (02:00:58).
- **Validação de Exceções no Front-end e API**: Márcio das Chagas Ramos levantou a questão do tratamento de múltiplas exceções em formulários, onde o front-end geralmente lida com validações como o tamanho dos campos. Caso o front-end não realize essas validações, a API deve fazê-lo, especialmente quando o dado não provém do front-end, mas de uma fila, por

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

exemplo. Nesse cenário, a resposta ideal para quem chamou a API seria listar todos os tipos de exceções encontradas de uma só vez (02:03:23).

- **Progressão e Evolução das Validações**: Francisco Schneider e Márcio das Chagas Ramos discutiram que as validações mais simples são realizadas no front-end e na camada de borda (usando ferramentas como C# ou FluentValidation). As validações de negócio, por sua vez, são aplicadas no domínio antes de materializar ou criar a entidade, e essas validações falharão se o dado estiver incorreto (02:04:18). Essa evolução das validações, conforme se adentra no domínio, garante uma cobertura mais completa na criação do objeto (02:05:30).
- **Limitações do Uso Apenas de Exceções (Exceptions)**: Francisco Schneider apresentou as limitações do uso exclusivo de exceções, citando um método cujo retorno (nulo ou exceção) não é claro pela assinatura. Eles notaram que o cliente do código (desenvolvedor) não consegue entender o comportamento esperado (se devem tratar nulo, usar `try-catch`) sem analisar a implementação. Isso é especialmente verdadeiro em casos de falha de conexão com o banco de dados (02:05:30).
- **O Brilho do Result Pattern**: Francisco Schneider introduziu o Result Pattern como uma solução para tornar os métodos mais significativos, o que aborda pontos levantados por Márcio das Chagas Ramos e Gustavo Ferraz Fontes. Uma boa implementação de Result deve ser genérica e pode utilizar operadores implícitos para facilitar o retorno de sucesso ou falha, em vez de retornar apenas um valor booleano ou uma string (02:06:52). O uso de um objeto `Result` mais complexo é necessário quando o tratamento de erros requer diferenciar entre tipos de erro (erro X versus erro Y) (02:08:16).
- **Uso da Biblioteca C# Fun Extensions e Maybe**: Francisco Schneider mencionou a biblioteca C# Fun Extensions, que oferece recursos funcionais e envelopes de resultados (02:08:16). Eles discutiram o uso do `Maybe` dentro do contexto do Result Pattern, que é mais expressivo ao lidar com operações que podem ou não retornar um dado (como recuperar um dado no banco de dados). O uso de `Maybe` é considerado mais expressivo do que retornar um `professional?` (nullable), pois obriga o desenvolvedor a verificar o retorno, evitando que eles usem diretamente o valor sem checagem (02:09:36) (02:12:14).
- **Result Pattern, Exceptions e Maturidade do Time**: Francisco Schneider destacou que a escolha entre Result Pattern e Exceptions, apesar de eles terem preferência por Result Pattern, muitas vezes depende da maturidade e familiaridade da equipe. Embora o Result Pattern torne o código mais legível e fácil de entender, as equipes podem preferir usar exceções devido à

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

familiaridade, mesmo que o uso correto das exceções seja complexo (02:13:04) (02:20:17). Eles observaram que é mais comum encontrar implementações problemáticas com exceções do que com Result Pattern (02:21:24).

- **Comparação de Custo e Medição de Performance**: Eduardo Kubota perguntou sobre como medir o custo de uma exceção e quando escolher entre Result Pattern e Exception (02:15:38). Francisco Schneider respondeu que a medição de performance e uso de recursos pode ser feita com testes de carga e teste de estresse, ou um benchmark para casos mais simples (02:16:50) (02:19:08). Eles argumentaram que a diferença de performance entre o uso de Result e Exception é insignificante para o negócio, sendo a legibilidade e a semântica as principais vantagens do Result Pattern (02:20:17).
- **Decisão de Implementação entre Result e Exception**: Francisco Schneider orientou que Result Pattern é adequado para a maioria dos casos, enquanto Exceptions devem ser reservadas para cenários específicos onde se deve "quebrar tudo" ou interromper o *pipeline* (02:19:08). A decisão final sobre qual usar (ou a combinação de ambos) depende da familiaridade e maturidade da equipe, e não de uma regra técnica absoluta (02:24:21).
- **O Conceito de Maybe como Abstração para Null Pattern**: Farley Souza perguntou se o `Maybe` seria uma abstração do Result Pattern (02:29:04). Francisco Schneider esclareceu que o `Maybe` se complementa ao Result Pattern e é usado para situações em que a ausência de um resultado não é necessariamente uma falha, enriquecendo a semântica do retorno (02:30:33). O `Maybe` é uma estrutura leve que permite dizer que "talvez eu retorne" um dado, sendo útil em cenários como repositórios (02:31:36).
- **Princípio da Responsabilidade Única (SRP)**: Francisco Schneider iniciou a discussão sobre o SOLID com o SRP, definindo-o como o princípio de que uma classe deve ter apenas uma razão para mudar. Eles destacaram que o foco não é contar o número de métodos, mas identificar os "atores" ou o "dono" da regra que pode causar a mudança (02:32:30). A violação ocorre quando a classe tem múltiplas razões para mudar, como um `RegistrationService` que lida com registro, validação e cancelamento (02:33:38).
- **Sinais de Violação do SRP e Melhoria**: O SRP visa o controle de mudanças e a redução do *blast radius* (impacto das mudanças) (02:33:38). Sinais de alerta (bad smells) para a violação do SRP incluem classes que servem a múltiplas camadas (negócio, infraestrutura, banco de dados) e a injeção excessiva de serviços (02:34:54). A melhoria na implementação do SRP é

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

alcançada delegando responsabilidades a classes específicas e permitindo que o *manager* ou *handler* se concentre na orquestração, o que facilita os testes (02:36:16).

- **Granularização de Handlers para SRP**: Natan Barros perguntou como evitar a violação do SRP em *handlers* que crescem com mais operações (02:37:42). Francisco Schneider sugeriu a criação de um *handler* ou *manager* por operação (`CreateOrderManager`, `CancelOrderManager`), em vez de um único `OrderManager` (02:39:01). Essa granularização alinha-se com a ideia de ter um *handler* para cada comando e caso de uso, e melhora a performance ao injetar apenas os serviços necessários (02:40:21).
- **Princípio Aberto/Fechado (OCP)**: O OCP estabelece que o código deve ser aberto para extensão e fechado para modificação. A violação ocorre quando uma nova funcionalidade exige mexer no código central (como um `switch` ou `if` existente) (02:45:19). Francisco Schneider demonstrou como utilizar um dicionário (*read-only dictionary*) para mapear diferentes implementações de processadores de pagamento, eliminando a necessidade de alterar o `PaymentService` toda vez que um novo tipo de pagamento surgir (02:46:39).
- **Discussão sobre o Padrão Mediator e OCP**: Farley Souza questionou se o padrão Mediator seria uma boa referência para o OCP (02:48:08). Francisco Schneider explicou que, embora o Mediator possa ser usado, ele ainda exige que a camada de disparo (como um Controller) seja alterada para criar o comando correto, o que viola o OCP. Eles sugeriram o padrão Decorator como um exemplo que atende bem ao OCP, pois permite adicionar funcionalidade (como *caching*) sem modificar a classe existente (02:49:39).
- **Princípio da Substituição de Liskov (LSP)**: O LSP foi apresentado como o princípio mais complexo, exigindo que as subclasses sejam intercambiáveis sem alterar o comportamento em tempo de execução (02:50:57). A violação do LSP (percebida apenas em *runtime*) ocorre quando uma classe filha não cumpre a promessa da classe pai, como lançar uma exceção não esperada ou retornar um resultado "mais fraco" (como um usuário vazio) (02:52:17). A violação é comum quando a herança é usada primariamente para reaproveitamento de código (02:53:45).
- **Garantia do LSP e Confiança**: Francisco Schneider afirmou que o LSP se resume à "confiança" e "previsibilidade" (02:52:17) (02:56:39). Se o código cliente precisar se preocupar com qual implementação concreta está sendo utilizada, o LSP foi violado. Para promover o LSP, o contrato deve ser explícito e rico em detalhes sobre os resultados esperados (02:55:10).

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

- **Princípio da Segregação de Interfaces (ISP)**: O ISP, o último princípio abordado, visa evitar que os clientes sejam obrigados a implementar métodos em interfaces que não utilizam (02:56:39).
- **Princípio de Segregação de Interfaces (Interface Segregation Principle - ISP)**: Francisco Schneider discutiu o princípio ISP, destacando problemas que surgem ao usar interfaces grandes ou "significativas" por razões de padronização ou reaproveitamento. Ele mencionou que, em implementações concretas, os desenvolvedores são obrigados a implementar todos os métodos, mesmo aqueles que não são necessários para o cenário específico, levando a perdas de produtividade e dificuldades em testes, devido à necessidade de mais *mocks* (02:58:04). A solução ideal sugerida é pensar nas interfaces como "papéis" (*roles*) ou habilitadores, em vez de "espelhos das classes", permitindo que uma classe implemente múltiplas interfaces menores, cada uma focada em um conjunto específico de comportamentos (leitura, escrita, controle transacional, etc.) (02:59:20).
- **Vantagens de Interfaces Segregadas e Discussão sobre Herança de Interfaces**: Francisco Schneider destacou que as vantagens das interfaces segregadas incluem a melhoria da produtividade, pois o desenvolvedor implementa apenas o que é necessário, o que, por sua vez, reduz a complexidade e facilita a manutenibilidade (03:02:07). Natan Barros perguntou se a abordagem de ter uma interface genérica que herda de todas as interfaces menores necessárias para uma classe específica (agregador) seria uma prática aceitável; Francisco Schneider respondeu que isso é considerado aceitável, mas o problema principal reside em ter uma "super interface" com muitos métodos ou expor a implementação completa a clientes que só precisam de pequenas fatias dela (03:00:42).
- **Princípio da Inversão de Dependência (Dependency Inversion Principle - DIP)**: Francisco Schneider introduziu o Princípio da Inversão de Dependência com a regra de que abstrações não devem depender de detalhes, mas sim que os detalhes (implementações) dependem de abstrações (03:02:07). O domínio ou as regras de negócio não devem ter conhecimento direto de *frameworks*, bancos de dados, serviços HTTP ou ORMs; esse conhecimento deve ser resolvido externamente por meio de contratos e injeção de dependência. Francisco Schneider ressaltou que a criação manual de instâncias de clientes HTTP ou o acoplamento direto a implementações concretas é um erro comum e viola o princípio DIP, além de causar problemas de performance (03:03:24).
- **Benefícios do DIP e Conclusão da Reunião**: Os benefícios de se aderir ao DIP incluem a diminuição do acoplamento, o favorecimento da escrita de testes e o aumento da flexibilidade do código, uma vez que o código depende de

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

abstrações e não de implementações concretas. Francisco Schneider concluiu a discussão sobre os princípios SOLID e orientou que a complexidade dos tópicos aumentará nas próximas sessões de nivelamento, com o objetivo de discutir como criar soluções orientadas a objetos dentro do *framework*. Eles encerraram a reunião e incentivaram a continuação das discussões no grupo de WhatsApp (03:04:52).

Revise as anotações do Gemini para checar se estão corretas. Confira dicas e saiba como o Gemini faz anotações

*Como está a qualidade de **destas observações?** Responda a uma breve pesquisa para nos dar seu feedback, incluindo o quanto as observações foram úteis para o que você precisa.*

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com